

Модуль 9. От очередей к событиям: архитектурные отличия и рождение Kafka

9.1. Зачем мы здесь: от команды к факту

В предыдущих восьми модулях мы построили прочный фундамент. Мы научились упаковывать приложения в Docker, собирать многоэтапные образы, поднимать связки сервисов через Compose, работать с Redis Streams и, наконец, организовывать фоновые задачи через Celery. Мы знаем, как поставить задачу в очередь, как воркер её забирает, как подтверждается выполнение.

Но теперь перед нами встаёт новый вопрос. Представьте, что ваш ML-сервис растёт. У вас уже не один FastAPI + Celery. У вас появляются:

- Сервис аналитики, который хочет знать, сколько моделей обучилось за сегодня.
- Сервис мониторинга, который хочет отслеживать каждый запрос на предсказание.
- Сервис аудита, который должен хранить историю всех действий пользователя по закону.
- Новый микросервис, которому нужны те же данные, что и старому, но в реальном времени.

Если вы попытаетесь решить это всё через Celery и Redis Lists, вы столкнётесь со стеной. Почему? Потому что Celery построен на **парадигме очереди задач**, а растущая система требует **парадигмы лога событий**. Этот модуль посвящён тому, чтобы вы почувствовали эту разницу внутри, а не просто заучили определения.

9.2. Две парадигмы: «Сделай это» vs «Произошло это»

Чтобы понять разницу между очередью задач и логом событий, нужно различать два типа мышления. Они настолько фундаментально разные, что использование одного инструмента для другого — всё равно что забивать гвозди микроскопом.

9.2.1. Парадигма очереди задач (Task Queue): императив

Суть: «Эй, ты! Выполни вот это действие!»

Сообщение в очереди задач — это **команда**. Оно говорит получателю, что именно ему нужно сделать. Получатель выполняет команду, и сообщение теряет смысл. Его можно выбросить.

Аналогия: приказ в армии

Сержант кричит рядовому: «Отожмись 20 раз!» Рядовой отжимается. Приказ выполнен. Бумажка с приказом сжигается. Нет никакой необходимости хранить её в архиве. Если

рядовой спросит через неделю: «А что мне вчера приказывали?» — никто не будет лезть в мусорку за сожжённой бумажкой. Приказ был эфемерен.

В коде (Celery):

```
In [1]: # Это КОМАНДА
train_model.delay("dataset.csv", "xgboost")
# Смысл: "Воркер, запусти функцию train_model с этими аргументами"
```

Характеристики:

- **Эфемерность:** Сообщение существует только до выполнения.
- **Цель:** Изменить состояние мира (обучить модель, отправить письмо).
- **Один исполнитель:** Одна задача обычно выполняется одним воркером.
- **Нет истории:** После ACK сообщение исчезает навсегда.

9.2.2. Парадигма лога событий (Event Log): декларативная

Суть: «Вот что произошло. Запиши это. Кто хочет — реагируй.»

Сообщение в логе событий — это **факт**. Оно не приказывает никому ничего делать. Оно просто констатирует: «В 14:32:15 пользователь с ID 42 загрузил файл report.csv». Этот факт интересен многим системам. И он остаётся фактом навсегда.

Аналогия: газета или судовой журнал

Капитан пишет в судовом журнале: «20 августа, 14:32. Видели китов на широте 45°.» Это не приказ кому-то. Это фиксация реальности. Этот журнал может прочитать:

- Адмиралтейство (чтобы проверить маршрут).
- Учёный-биолог (чтобы изучить миграцию китов).
- Страховая компания (чтобы подтвердить, что корабль был в этом районе).
- Потомки (чтобы написать книгу).

Журнал не говорит: «Эй, биолог, иди изучай китов!» Он просто существует. Читатели сами решают, как на него реагировать.

В коде (Kafka):

```
In [2]: # Это ФАКТ
producer.send("user.events", {
    "timestamp": "2026-08-20T14:32:15Z",
```

```
"user_id": 42,  
"action": "file_uploaded",  
"filename": "report.csv"  
})  
# Смысл: "Это произошло. Кому надо — прочитайте."
```

Характеристики:

- **Персистентность:** Сообщение хранится долго (дни, месяцы, годы).
- **Цель:** Зафиксировать факт для последующего анализа.
- **Множественные читатели:** Десятки независимых сервисов читают один поток.
- **История:** Можно «перемотать» назад и прочитать события недельной давности.

9.3. Эфемерные vs персистентные сообщения: анатомия различий

Давайте разберём каждое свойство под микроскопом.

9.3.1. Время жизни сообщения

Очередь задач (Celery + Redis List): Сообщение живёт от момента отправки до момента АСК. Обычно это секунды или минуты. После успешного выполнения оно стирается из Redis. Если вы хотите узнать, что происходило вчера — вы не можете. Истории нет.

Аналогия: Заказ в ресторане быстрого питания. Чек печатается, отдаётся на кухню, готовится, выдаётся клиенту, чек в мусорку. Через час никто не скажет, что заказывал столик №5.

Лог событий (Kafka / Redis Streams): Сообщение живёт столько, сколько настроена политика хранения. Это может быть 7 дней, 30 дней, год или «навсегда» (пока не закончится диск). Сообщение не удаляется после прочтения.

Аналогия: Газетный архив в библиотеке. Газеты за 1920 год всё ещё лежат на полках. Вы можете прийти и прочитать.

9.3.2. Кто читает

Очередь задач: Одно сообщение — один исполнитель. Если воркер А взял задачу, воркер Б её не получит. Это критично: мы не хотим, чтобы одно и то же письмо отправилось дважды.

Аналогия: Билет в кино. Один билет — одно место. Если Иван купил билет, Мария не может сесть на то же место.

Лог событий: Одно сообщение — любое количество читателей. Каждый читатель читает независимо, со своей скоростью, со своей позиции.

Аналогия: Подкаст. Один выпуск. Миллион слушателей. Каждый слушает в своё время, на своей скорости. Подкаст не исчезает после того, как его послушал первый человек.

9.3.3. Что содержит сообщение

Очередь задач: Сообщение содержит **команду**: имя функции, аргументы, kwargs. Это инструкция к действию.

Пример (внутренний формат Celery):

```
In [3]: {
  "task": "tasks.train_model",
  "args": ["dataset.csv", "xgboost"],
  "kwargs": {}
}
```

Лог событий: Сообщение содержит **факт**: что произошло, когда, кто участвовал, какие атрибуты. Это не инструкция, это запись.

Пример (Kafka):

```
In [4]: {
  "event_type": "model_trained",
  "timestamp": "2026-08-20T14:32:15Z",
  "model_id": "uuid-1234",
  "accuracy": 0.94,
  "dataset_version": "v2.1"
}
```

9.3.4. Что делает Consumer

Очередь задач: Consumer **изменяет мир**. Он обучает модель, отправляет email, генерирует PDF. Его работа имеет побочные эффекты.

Лог событий: Consumer **реагирует на факт**. Он может:

- Обновить дашборд.
- Отправить уведомление.
- Записать в другую базу данных.
- Ничего не делать, просто накопить статистику.

Важно: сам факт уже произошёл. Consumer не может его отменить. Он только обрабатывает последствия.

9.4. Почему Celery/Redis не подходит для потоковой аналитики и репликации

Теперь, когда мы разобрали парадигмы, давайте конкретно поймём, где именно Celery задыхается, и почему там нужна Kafka.

9.4.1. Проблема 1: «Хочу добавить новый сервис, но задачи уже сожжены»

Представьте: у вас есть система, где при загрузке датасета Celery-воркер обучает модель. Всё работает. Потом бизнес говорит: «Нам нужен новый сервис, который при каждом обучении модели обновляет рекомендации в интернет-магазине.»

C Celery: Вы вынуждены лезть в код воркера `train_model` и добавлять туда вызов нового сервиса. Или создавать новую задачу, которую Producer должен отправлять параллельно. Но что со старыми задачами? Они уже выполнены, исчезли. Новый сервис никогда не узнает, что модели обучались вчера.

C Kafka: Вы просто создаёте новый Consumer Group и подключаете её к топике `model.events`. Новый сервис начинает читать события с момента подключения (или с начала, если нужна история). Старый сервис продолжает работать. Producer вообще не знает о существовании нового сервиса. Это называется **слабая связанность** (loose coupling).

9.4.2. Проблема 2: «Хочу перемотать на вчера»

Бизнес спрашивает: «Покажите, сколько пользователей загрузило датасеты в прошлый понедельник с 14:00 до 15:00.»

C Celery: Невозможно. Задачи выполнены, ACK отправлены, Redis их забыл. У вас нет журнала.

C Kafka: Возможно. Вы создаёте временного Consumer'a, указываете timestamp (понедельник, 14:00) и читаете события из топика за этот час.

9.4.3. Проблема 3: «Нужна репликация данных между сервисами»

У вас есть сервис заказов (PostgreSQL) и сервис аналитики (ClickHouse). Нужно, чтобы данные из заказов попадали в аналитику с задержкой не более 1 секунды.

C Celery: Вы создаёте задачу «скопировать заказ в ClickHouse». Но что если задача упала? Что если заказ обновился? Как синхронизировать DELETE? Это превращается в ад хаков.

С Kafka: Вы используете Change Data Capture (CDC). База данных пишет изменения в Kafka. Сервис аналитики читает их как поток событий. Это промышленный стандарт.

9.4.4. Проблема 4: «Нужен аудит по закону»

Закон требует: храните историю всех действий пользователей 5 лет.

С Celery: Невозможно. Celery не хранит историю.

С Kafka: Настраиваете retention на 5 лет (или больше, с Tiered Storage). Всё хранится.

9.5. Где Kafka незаменима: конкретные сценарии в ML

Давайте привяжем абстракции к реальной жизни ML-инженера.

9.5.1. Поточковая предобработка данных (Stream Processing)

Представьте: пользователь загружает фото товара. Нужно:

1. Сохранить оригинал.
2. Сжать до thumbnail'a.
3. Извлечь признаки нейросетью.
4. Записать признаки в векторную базу данных.
5. Обновить поисковый индекс.

С Kafka: Каждый шаг — это отдельный сервис, читающий топик и пишущий в другой топик.

- Топик `raw.images` -> сервис сжатия -> Топик `thumbnails.ready` -> сервис извлечения признаков -> Топик `features.extracted` -> сервис индексации.

Каждый сервис работает независимо, масштабируется отдельно. Если сервис извлечения признаков тормозит — он просто отстаёт от потока, но не ломает остальных. Это **конвейер** (pipeline).

С Celery: Вы создаёте одну огромную задачу, которая делает всё подряд. Или цепочку задач (`chain`), но управлять таким конвейером сложно, и нет прозрачности — вы не видите, на каком этапе «застряло» сообщение.

9.5.2. ML-пайплайны: обучение и инференс в реальном времени

Представьте рекомендательную систему онлайн-магазина. Пользователь кликает по товарам. Нужно мгновенно обновлять рекомендации.

С Kafka: Поток кликов (`user.clicks`) поступает в Kafka. Модель читает этот поток, обновляет векторы пользователей в реальном времени. Рекомендации обновляются за миллисекунды.

С Celery: Вы создаёте задачу на каждый клик. Но кликов 10 000 в секунду. Celery-воркеры захлебнутся в накладных расходах (сериализация, десериализация, polling Redis). Redis List не оптимизирован для такого потока.

9.5.3. Аудит событий и compliance

В медицинских ML-системах (диагностика по рентгену) каждое предсказание модели должно быть задокументировано: кто загрузил снимок, какая модель, какая версия, какой результат, какой врач подтвердил.

С Kafka: Каждое действие — событие в топике `medical.audit` . Хранится 10 лет. Любой аудитор может прийти и проверить.

С Celery: Вы можете создать задачу «записать в аудит», но если воркер упал — аудит потерян. И нет централизованного журнала.

9.5.4. Репликация данных между командами

Команда А обучает модель и хранит метаданные в своей PostgreSQL. Команда В (фронтенд) хочет показывать пользователю статус обучения. Команда С (аналитика) хочет строить отчёты.

С Kafka: Команда А публикует событие `model.training_completed` . Команды В и С читают один и тот же топик. Команда А не знает о существовании В и С. Она просто фиксирует факт.

С Celery: Команда А должна явно вызывать API команды В и API команды С. Или создавать задачи для их воркеров. Это создаёт жёсткую связанность (tight coupling). Если команда В недоступна — задача падает.

9.6. Архитектурные отличия: таблица противопоставлений

Чтобы закрепить понимание, разберём каждый аспект в деталях.

9.6.1. Push vs Pull

Celery (Push): Producer толкает (push) задачу в брокер. Worker пассивно ждёт у брокера. Брокер сам решает, какому Worker'у отдать задачу.

Аналогия: Официант несёт заказ на кухню и кладёт в стопку. Повар берёт сверху. Официант не знает, какой именно повар возьмёт.

Kafka (Pull): Producer пишет (append) в лог. Consumer сам решает, когда и сколько сообщений прочитать. Он «тянет» (pull) данные из Kafka в своём темпе.

Аналогия: Читатель приходит в библиотеку и сам берёт книги с полки. Библиотекарь не бежит за читателем с книгами. Читатель читает в своём темпе: быстро или медленно. Может остановиться. Может вернуться к предыдущей главе.

Почему Pull лучше для потоков: Потому что разные Consumer'ы имеют разную скорость. Один — быстрый сервис фильтрации. Другой — медленный сервис машинного обучения, который обрабатывает одно сообщение 5 секунд. В модели Push медленный Consumer тормозил бы всю систему. В модели Pull он просто отстаёт, не мешая остальным.

9.6.2. Централизованное состояние vs Распределённый лог

Celery + Redis: Redis — единая точка хранения. Все задачи лежат в одном списке (или нескольких очередях). Redis должен держать всё в памяти. Если Redis упал — система остановилась (если нет репликации).

Kafka: Данные распределены по множеству серверов (брокеров). Каждый топик разбит на партиции. Данные хранятся на диске, а не только в памяти. Kafka спроектирована так, что отказ одного брокера не останавливает систему.

9.6.3. Упорядоченность

Celery: Гарантирует порядок только в пределах одной очереди и только теоретически. Если у вас 8 воркеров, задачи выполняются параллельно, и порядок завершения непредсказуем.

Kafka: Гарантирует строгий порядок внутри одной партиции. Если событие А было записано раньше события В в партицию 0, то все Consumer'ы увидят А раньше В. Это критично для финансовых операций и event sourcing.

9.7. Практика: сравнительная таблица «Celery или Kafka»

Критерий	Celery (Очередь задач)	Kafka (Лог событий)
Природа сообщения	Команда: «Сделай X»	Факт: «Произошло Y»
Время жизни	Секунды/минуты (до ACK)	Дни/месяцы/годы
Цель	Выполнить действие	Зафиксировать событие
Читатели	Ровно один воркер	Любое количество независимых групп
История	Нет	Полная, можно перемотать
Модель доставки	Push (брокер толкает воркеру)	Pull (consumer сам читает)

Критерий	Celery (Очередь задач)	Kafka (Лог событий)
Хранение	В памяти (Redis)	На диске (файловая система)
Масштабирование	Добавлять воркеров	Добавлять партиции и брокеров
Порядок	Не гарантирован при параллелизме	Гарантирован внутри партиции
Аудит/Compliance	Невозможен	Встроенный
Потоковая обработка	Неэффективна	Родная среда
Добавление нового сервиса	Нужно менять Producer	Новый Consumer читает существующий топик
Сложность	Низкая	Средняя/Высокая
Типичный use-case	Отправка email, обучение модели по запросу, генерация отчётов	Реалтайм-аналитика, репликация данных, event sourcing, аудит

9.8. Практика: выбираем технологию для сценариев

Давайте потренируемся. Прочитайте сценарий и решите: Celery или Kafka? Потом проверьте ответ.

Сценарий 1

«Пользователь нажал кнопку 'Сгенерировать годовой отчёт в PDF'. Отчёт строится 3 минуты. Пользователь должен получить email, когда отчёт готов.»

Ответ: Celery. Это классическая фоновая задача. Нужно выполнить действие, получить результат, уведомить пользователя. История не нужна. Одно письмо — один воркер.

Сценарий 2

«Пользователь кликает по товарам на сайте. Нужно в реальном времени обновлять рекомендации. Кликов 50 000 в секунду. Также аналитики хотят строить воронку продаж за прошлый месяц.»

Ответ: Kafka. Потоковая обработка, высокая нагрузка, нужна история для аналитики, множество потребителей (рекомендации + аналитика).

Сценарий 3

«При обучении модели нужно зафиксировать: кто запустил, какие гиперпараметры, какая accuracy, кто одобрил деплой. Эти данные нужны аудиторам через 3 года.»

Ответ: Kafka (или Redis Streams для простых случаев). Нужен неизменяемый аудит-лог. Celery не сохраняет историю.

Сценарий 4

«Ночью запускается пакетное обучение 100 моделей. Каждая модель — отдельная задача. Нужно распределить нагрузку на 10 GPU-серверов.»

Ответ: Celery. Массовое распределение независимых задач по воркерам — это классика очередей задач. GPU-серверы = воркеры.

Сценарий 5

«Есть сервис заказов. Нужно, чтобы данные о новых заказах мгновенно попадали в поисковый индекс, в CRM, в складскую систему и в аналитику.»

Ответ: Kafka. Один факт (новый заказ) -> четыре независимых Consumer'а. Слабая связанность.

9.9. Переходный мост: от Redis Streams к Kafka

Вы можете спросить: «А почему бы просто не использовать Redis Streams для всего? Ведь это тоже лог событий!»

Действительно, Redis Streams — это шаг в сторону логов событий. Но Redis имеет фундаментальные ограничения:

- **Память:** Redis хранит данные в оперативной памяти (или частично на диске через AOF, но это не то же самое). Хранить терабайты событий дорого.
- **Масштабирование:** Redis — это один сервер (или master-replica). Kafka — это распределённая система из многих брокеров, способная обрабатывать миллионы сообщений в секунду.
- **Персистентность:** Kafka спроектирована так, что данные живут на диске и читаются с диска последовательно — это очень быстро. Redis оптимизирован для случайного доступа в памяти.

Redis Streams хорош для:

- Небольших систем (до тысяч сообщений в секунду).

- Систем, где уже есть Redis и не хочется добавлять Kafka.
- Временного буфера между сервисами.

Kafka необходим для:

- Больших данных (Big Data).
- Долгосрочного хранения.
- Множества независимых команд, читающих одни данные.
- Геораспределённых систем.

9.10. Итоги модуля: чек-лист

- ☐ Понимаю фундаментальную разницу между **командой** (очередь задач) и **фактом** (лог событий).
- ☐ Знаю, что очередь задач **эфемерна**: сообщение исчезает после выполнения.
- ☐ Знаю, что лог событий **персистентен**: сообщения хранятся долго и неизменны.
- ☐ Понимаю, почему Celery не подходит для потоковой аналитики, аудита и репликации.
- ☐ Знаю, что Kafka незаменима для: ML-пайплайнов, real-time аналитики, event sourcing, аудита.
- ☐ Понимаю разницу **Push** (Celery) и **Pull** (Kafka) моделей доставки.
- ☐ Знаю, что в Kafka множество независимых Consumer Groups читают один топик, а в Celery одна задача — один воркер.
- ☐ Умею выбирать между Celery и Kafka для конкретного сценария.
- ☐ Понимаю, почему Redis Streams — это промежуточный вариант, но не замена Kafka для больших данных.

В следующем модуле мы наконец познакомимся с **Apache Kafka** — её архитектурой, топиками, партициями, офсетами и группами потребителей. Мы развернём Kafka через Docker Compose и начнём работать с ней через консольные утилиты.